

Overall Burndown Analysis

In terms of a burndown chart, we believe there are two ways to look at our progress.

In terms of the exact sprint 3 requirements:

- We started ahead of schedule because of the work we did in sprint 2
- At around halfway we had about half our work remaining
 - We did get an A on the functionality check-in though
- Near the end, we were behind, rushing to get in the specific level requirements

In terms of the game as a whole:

- We are constantly coming up with new ideas, which usually always increase the amount of hours to completion, including the amount of time for each sprint's requirements
 - This is partly because the original Starseed Pilgrim is very different from the traditional guidelines of this course
 - This is also because we keep having new ideas of how to incorporate the requirements into something that suits our ideas
 - Also, some of these ideas actually decrease time to completion. For example, the idea to make seeds the only usable item. They are used to plant plants, attack enemies, and propel yourself with knockback.
- Because of how many leads there still are to follow, there will always be more work to be done, even after this semester.

Big Changes

Constructive:

- As mentioned above, having seed projectile as the only item simplified much of the inventory logic, and provides a unique constraint to work under
- Switch to momentum-based collision allowed us to implement entities having knockback from shooting projectiles, getting knockback from being hit by projectiles, detecting when entities are grounded, and detecting when a seed can be planted all within one framework

Detrimental:

- Short-term, switching to momentum-based collision took a disproportionate amount of time and could be seen as somewhat of a premature optimization
- There were a few back-and-forth discussions about the best way to implement the above, as well as fixing many of the bugs that the above change introduced
- Using a CSV to store level data. We did this basically just to hit the requirement, but in the long run we want everything to be randomly generated a la Minecraft terrain.
 - However, once we have more complex level states, this could be useful to store save files, though again we're still deciding on what file format would work best